Aprendizaje de Máquina
Sesión 16: Reducción de dimensiones y Aprendizaje de máquina para textos

Angel Rodríguez

PCA en Scikit-Learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)
X_reduced = pca.fit_transform(X_train)

X_recovered = pca.inverse_transform(X_reduced)
```

- `fit_transform()` reduce la dimensionalidad.
- `inverse_transform()` reconstruye una aproximación.
- La reconstrucción suele ser parecida, pero con cierta pérdida de detalle.

Randomized PCA

Para datasets grandes, calcular PCA con SVD completa puede ser costoso.

Randomized PCA usa un algoritmo estocástico para aproximar los primeros d componentes principales.

Ventaja principal:

$$O(md^2) + O(d^3)$$

en lugar de aproximadamente

$$O(mn^2) + O(n^3)$$

Es especialmente útil cuando:

$$d \ll n$$

Randomized PCA en código

```
from sklearn.decomposition import PCA

rnd_pca = PCA(
    n_components=154,
    svd_solver="randomized",
    random_state=42
)

X_reduced = rnd_pca.fit_transform(X_train)
```

En Scikit-Learn, `svd_solver="auto"` (default) decide automáticamente qué método usar.

Si queremos forzar SVD completa:

```
pca = PCA(n_components=154, svd_solver="full")
```

Incremental PCA

Una limitación de PCA estándar es que necesita que todo el dataset esté en memoria.

Incremental PCA permite procesar los datos por mini-batches.

Esto es útil cuando:

- El dataset es demasiado grande para caber en memoria.
- Queremos aplicar PCA de forma incremental.
- Los datos llegan poco a poco.

Incremental PCA en código

```
from sklearn.decomposition import IncrementalPCA
import numpy as np

n_batches = 100
inc_pca = IncrementalPCA(n_components=154)

for X_batch in np.array_split(X_train, n_batches):
    inc_pca.partial_fit(X_batch)

X_reduced = inc_pca.transform(X_train)
```

La diferencia clave es usar:

`partial_fit()`

en lugar de entrenar con todo el dataset a la vez.

Random Projection

Cuando los datos tienen muchísimas dimensiones, PCA puede seguir siendo lento.

Random Projection proyecta los datos usando una matriz aleatoria:

$$X_{\text{reduced}} = XP^{\top},$$

donde cada entrada es generada con una distribución Gaussiana con media 0 y varianza $1/d$, con d el número de dimensiones a conservar.

Aunque suena sorprendente, una proyección aleatoria puede preservar bastante bien las distancias entre puntos.

Esto se justifica con el lema de Johnson–Lindenstrauss.

Número de dimensiones en Random Projection

La cantidad mínima de dimensiones depende de:

- m: número de instancias.
- ε: tolerancia máxima de distorsión.

Una cota típica es:

$$d \geq \frac{4\log(m)}{\frac{1}{2}\varepsilon^2 - \frac{1}{3}\varepsilon^3}$$

Observación importante:

d no depende directamente de n.

Random Projection en Scikit-Learn

```
from sklearn.random_projection import (
    johnson_lindenstrauss_min_dim,
    GaussianRandomProjection
)

m, eps = 5000, 0.1
d = johnson_lindenstrauss_min_dim(m, eps=eps)

gaussian_rnd_proj = GaussianRandomProjection(
    eps=eps,
    random_state=42
)

X_reduced = gaussian_rnd_proj.fit_transform(X)
```

Gaussian vs Sparse Random Projection

Scikit-Learn incluye dos variantes:

Método	Matriz aleatoria	Ventaja
GaussianRandomProjection	Densa	Simple y directa
SparseRandomProjection	Dispersa	Menos memoria, más rápida

Ambos métodos preservan aproximadamente las distancias, pero utilizan una matriz de proyección distinta.
Para datasets grandes o dispersos, suele convenir usar `SparseRandomProjection`.

LLE: Locally Linear Embedding

LLE es una técnica no lineal de reducción de dimensionalidad.

A diferencia de PCA o Random Projection:

LLE no se basa en una proyección lineal global.

La idea:

1. Para cada punto, encontrar sus vecinos más cercanos.
2. Expresar cada punto como combinación lineal de sus vecinos.
3. Buscar una representación de baja dimensión que preserve esas relaciones locales.

Intuición de LLE

LLE es especialmente útil cuando los datos están sobre una variedad no lineal.

Ejemplo clásico:

Swiss roll

LLE intenta “desenrollar” la variedad preservando relaciones locales.

Pero no siempre preserva bien las distancias globales.

LLE en código

```
from sklearn.datasets import make_swiss_roll
from sklearn.manifold import LocallyLinearEmbedding
import matplotlib.pyplot as plt

X_swiss, t = make_swiss_roll(n_samples=1000, noise=0.2, random_state=42)

lle = LocallyLinearEmbedding(n_components=2, n_neighbors=10, random_state=42)
X_unrolled = lle.fit_transform(X_swiss)

# Crear figura
fig = plt.figure(figsize=(12, 5))
# Swiss Roll original (3D)
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
ax1.scatter(X_swiss[:, 0],X_swiss[:, 1],X_swiss[:, 2],c=t,cmap="hot")
ax1.set_xlabel("$x_1$")
ax1.set_ylabel("$x_2$")
ax1.set_zlabel("$x_3$")
ax1.set_title("Swiss Roll original")
# Swiss Roll despus de LLE (2D)
ax2 = fig.add_subplot(1, 2, 2)
ax2.scatter(X_unrolled[:, 0],X_unrolled[:, 1],c=t,cmap="hot")
ax2.set_xlabel("$z_1$")
ax2.set_ylabel("$z_2$", rotation=0)
ax2.grid(True)
ax2.set_title("Swiss Roll usando LLE")

plt.tight_layout()
plt.show()
```

LLE: paso 1[1]

Para cada instancia $x^{(i)}$, buscamos pesos $w_{i,j}$ que reconstruyan el punto usando sus k vecinos más cercanos:

$$\hat{W} = \arg\min_{W} \sum_{i=1}^{m} \left\| x^{(i)} - \sum_{j=1}^{m} w_{i,j} x^{(j)} \right\|^2$$

sujeto a:

$$w_{i,j} = 0 \quad \text{si } x^{(j)} \text{ no es } k \text{ vecino más cercano de } x^{(i)}$$

$$\sum_{j=1}^{m} w_{i,j} = 1$$

[1] Para más detalles, consultar `https://arxiv.org/html/2011.10925v1`

LLE: paso 2[2]

Después se buscan puntos $z^{(i)}$ en baja dimensión que preserven esos pesos:

$$\hat{Z} = \arg\min_{Z} \sum_{i=1}^{m} \left\| z^{(i)} - \sum_{j=1}^{m} \hat{w}_{i,j} z^{(j)} \right\|^2$$

Es decir:

preservar relaciones locales $\Longrightarrow$ representación de baja dimensión

[2] Para más detalles, consultar `https://arxiv.org/html/2011.10925v1`

Otras técnicas de reducción de dimensionalidad

Técnica	Idea principal
`sklearn.manifold.`**`MDS`** (Multidimensional Scaling)	Reduce la dimensionalidad intentando preservar las distancias entre las instancias, buscando una representación de menor dimensión con distancias similares.
`sklearn.manifold.`**`Isomap`** (Grafo)	Conecta cada instancia con sus vecinos más cercanos para formar un grafo y preserva las distancias geodésicas, es decir, las distancias a lo largo del grafo.
t-SNE (t-distributed Stochastic Neighbor Embedding)	Mantiene juntas las instancias similares y separa las instancias diferentes; se utiliza principalmente para **visualización**, especialmente de clusters.
LDA (Linear Discriminant Analysis)	Método supervisado que busca los ejes que mejor separan las clases; estos ejes pueden utilizarse para reducir la dimensionalidad antes de clasificar.
UMAP (Uniform Manifold Approximation and Projection)	Técnica de reducción no lineal similar a t-SNE que intenta preservar tanto la estructura local como la estructura global de los datos. Para **visualización**.

Multidimensional Scaling (MDS)

`sklearn.manifold.MDS`

Idea del método

- Reduce la dimensionalidad intentando preservar las **distancias entre las instancias**.
- Busca una representación en menor dimensión en la que las distancias entre los puntos sean similares a las del espacio original.
- Por lo tanto, intenta conservar la **estructura geométrica global** de los datos.

¿En qué datos funciona mejor?

- Cuando las **distancias entre observaciones son informativas** y queremos conservarlas.
- Es especialmente útil con datos de dimensionalidad relativamente baja o moderada.
- Puede utilizarse para visualizar estructuras donde interesa conservar la geometría global de los datos.

Isomap

`sklearn.manifold.Isomap`

Idea del método

- Conecta cada instancia con sus **vecinos más cercanos** para formar un grafo.
- Después calcula las **distancias geodésicas**: distancias medidas siguiendo caminos sobre el grafo.
- Reduce la dimensionalidad intentando preservar estas distancias en lugar de las distancias euclidianas directas.

¿En qué datos funciona mejor?

- Cuando los datos se encuentran aproximadamente sobre una **variedad (manifold) suave y de baja dimensión**.
- Funciona particularmente bien cuando existe una **estructura global única**.
- Ejemplo típico: datos con forma de *Swiss roll*, donde la distancia euclidiana puede ser engañosa.

t-SNE

t-distributed Stochastic Neighbor Embedding

`sklearn.manifold.TSNE`

Idea del método

- Reduce la dimensionalidad intentando mantener **cerca las instancias similares**.
- Al mismo tiempo intenta mantener **separadas las instancias diferentes**.
- Da especial importancia a preservar la **estructura local** de los datos.

¿En qué datos funciona mejor?

- Datos de **alta dimensionalidad** que queremos visualizar en dos o tres dimensiones.
- Es especialmente útil cuando existen **grupos o clusters** que queremos identificar visualmente.
- Ejemplo: visualizar imágenes de MNIST en un mapa 2D.

Principalmente se utiliza para visualización, no como preprocesamiento para entrenar otro modelo.

Linear Discriminant Analysis (LDA)

`sklearn.discriminant_analysis.LinearDiscriminantAnalysis`

Idea del método

- A diferencia de los métodos anteriores, LDA es un método **supervisado**.
- Durante el entrenamiento busca los ejes que mejor **discriminan entre las clases**.
- Los datos se proyectan sobre estos ejes de manera que las distintas clases queden lo más separadas posible.

¿En qué datos funciona mejor?

- Cuando tenemos **etiquetas de clase**.
- Cuando nuestro objetivo es conservar principalmente la información que permite **distinguir unas clases de otras**.
- Puede utilizarse como reducción de dimensionalidad antes de entrenar otro algoritmo de clasificación.

UMAP

Uniform Manifold Approximation and Projection

Idea del método

- Es un método no lineal de reducción de dimensionalidad basado en la estructura de la variedad de los datos.
- Al igual que t-SNE, intenta conservar la **estructura local**.
- Sin embargo, UMAP también intenta conservar parte de la **estructura global** de los datos.

¿En qué datos funciona mejor?

- Datos de **alta dimensionalidad** que queremos visualizar.
- Es útil cuando nos interesa observar tanto **clusters locales** como la relación global entre ellos.
- Escala mejor que t-SNE a **datasets grandes**.

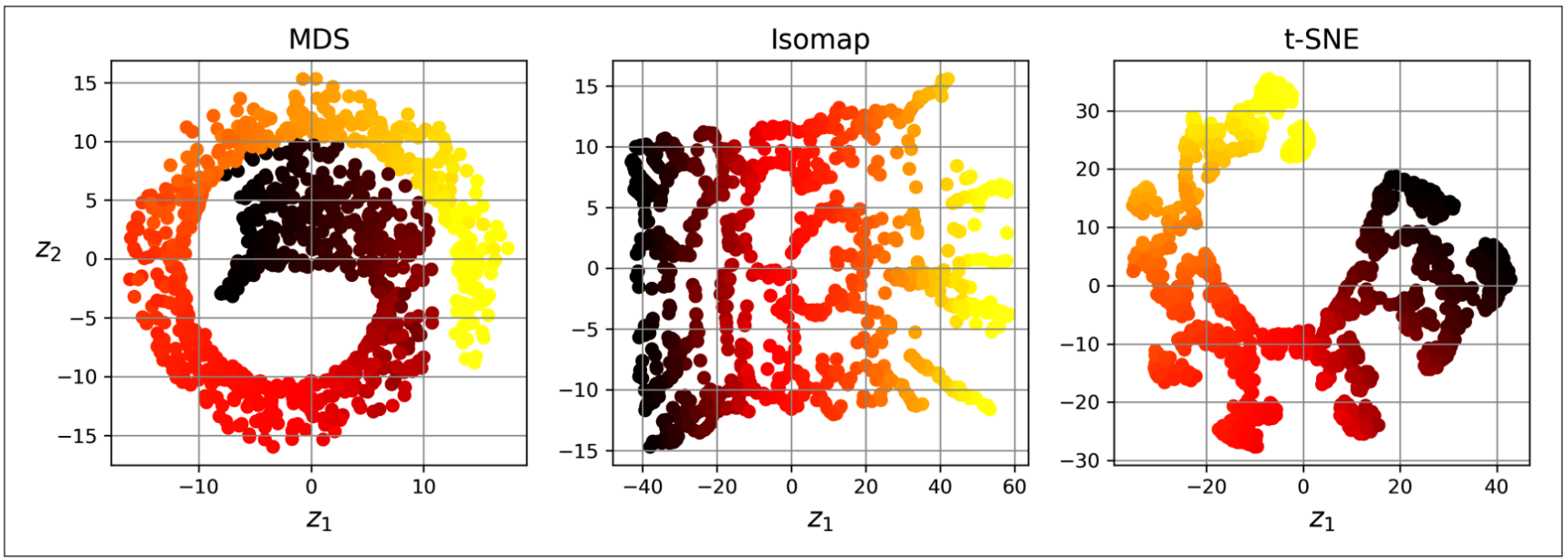
No está implementado directamente en Scikit-Learn; puede utilizarse mediante el paquete `umap-learn`.

Ejemplo Swiss roll

La figura compara MDS, Isomap y t-SNE sobre el *Swiss roll*:

- **MDS**: aplana los datos, pero conserva su curvatura global.
- **Isomap**: elimina la curvatura global.
- **t-SNE**: conserva algo de curvatura, pero enfatiza los clusters, llegando a separar el *Swiss roll*.

Qué representación es mejor depende de la tarea.



¿Cuándo usar cada técnica?

- **PCA**: reducción lineal, compresión y preprocesamiento.
- **Randomized PCA**: PCA más rápida cuando $d \ll n$.
- **Incremental PCA**: datasets grandes o por mini-batches.
- **Random Projection**: datos de altísima dimensión.
- **LLE**: variedades no lineales pequeñas o medianas.
- **t-SNE** / **UMAP**: visualización en 2D o 3D.

Resumen

- PCA puede usarse como técnica de compresión.
- La reconstrucción aproximada introduce error.
- Randomized PCA acelera el cálculo de componentes principales.
- Incremental PCA permite trabajar con datos que no caben en memoria.
- Random Projection es simple, rápida y útil en alta dimensión.
- LLE captura relaciones locales en datos no lineales.
- MDS, Isomap, t-SNE, LDA y UMAP son alternativas para distintos objetivos.

Sección 4. Aprendizaje de máquina para textos.

Utilizaremos principalmente el libro de

Jurafsky y Martin, *Speech and Language Processing*.

Tema 4.1: Introducción al procesamiento de textos

Hasta ahora hemos trabajado principalmente con datos que ya tienen una **representación numérica**:

$$x = (x_1, x_2, \ldots, x_p).$$

Tema 4.1: Introducción al procesamiento de textos

Hasta ahora hemos trabajado principalmente con datos que ya tienen una **representación numérica**:

$$x = (x_1, x_2, \ldots, x_p).$$

Sin embargo, un algoritmo de aprendizaje de máquina no puede utilizar directamente un documento como

"La película estuvo excelente".

Tema 4.1: Introducción al procesamiento de textos

Hasta ahora hemos trabajado principalmente con datos que ya tienen una **representación numérica**:

$$x = (x_1, x_2, \ldots, x_p).$$

Sin embargo, un algoritmo de aprendizaje de máquina no puede utilizar directamente un documento como

"La película estuvo excelente".

Necesitamos construir un flujo de procesamiento:

texto crudo $\longrightarrow$ preprocesamiento $\longrightarrow$ tokens $\longrightarrow$ representación numérica

En este tema estudiaremos principalmente las primeras etapas del proceso.

¿Qué es una palabra?

A primera vista, podríamos separar un texto utilizando los espacios.

```
El aprendizaje de máquina es interesante
```

¿Qué es una palabra?

A primera vista, podríamos separar un texto utilizando los espacios.

```
El aprendizaje de máquina es interesante
```

Pero aparecen inmediatamente algunas decisiones:

- ¿La puntuación constituye una unidad independiente?
- ¿Mayúsculas y minúsculas representan la misma palabra?
- ¿Cómo tratamos contracciones o palabras compuestas?
- ¿Qué ocurre en lenguas donde los espacios no marcan las fronteras entre palabras?

¿Qué es una palabra?

A primera vista, podríamos separar un texto utilizando los espacios.

```
El aprendizaje de máquina es interesante
```

Pero aparecen inmediatamente algunas decisiones:

- ¿La puntuación constituye una unidad independiente?
- ¿Mayúsculas y minúsculas representan la misma palabra?
- ¿Cómo tratamos contracciones o palabras compuestas?
- ¿Qué ocurre en lenguas donde los espacios no marcan las fronteras entre palabras?

Idea importante

La definición de “palabra” depende de la tarea y del lenguaje. Por ello necesitamos definir explícitamente las unidades que procesará el modelo.

Vocabulario: tipos e instancias

Consideremos:

"el gato mira al gato".

Podemos distinguir:

- **Instancias de palabras**: todas las apariciones en el texto.
- **Tipos de palabras**: las palabras distintas que aparecen.

Vocabulario: tipos e instancias

Consideremos:

"el gato mira al gato".

Podemos distinguir:

- **Instancias de palabras**: todas las apariciones en el texto.
- **Tipos de palabras**: las palabras distintas que aparecen.

En el ejemplo:

$$N = 5, \qquad V = \{\text{el, gato, mira, al}\}, \qquad |V| = 4.$$

Vocabulario: tipos e instancias

Consideremos:

"el gato mira al gato".

Podemos distinguir:

- **Instancias de palabras**: todas las apariciones en el texto.
- **Tipos de palabras**: las palabras distintas que aparecen.

En el ejemplo:

$$N = 5, \qquad V = \{\text{el, gato, mira, al}\}, \qquad |V| = 4.$$

Vocabulario

El conjunto de unidades distintas disponibles para representar el texto se denomina **vocabulario**. Su tamaño se denota normalmente por $|V|$.

Esta idea será fundamental cuando transformemos documentos en vectores.

El problema de las palabras desconocidas

Supongamos que construimos el vocabulario utilizando los datos de entrenamiento:

$$V = \{\texttt{low}, \texttt{new}, \texttt{newer}, \ldots\}.$$

Después aparece en un nuevo documento:

$$\texttt{lower}.$$

El problema de las palabras desconocidas

Supongamos que construimos el vocabulario utilizando los datos de entrenamiento:

$$V = \{\texttt{low}, \texttt{new}, \texttt{newer}, \ldots\}.$$

Después aparece en un nuevo documento:

$$\texttt{lower}.$$

Si utilizamos exclusivamente palabras completas, `lower` no pertenece al vocabulario.

$$\texttt{lower} \notin V.$$

El problema de las palabras desconocidas

Supongamos que construimos el vocabulario utilizando los datos de entrenamiento:

$$V = \{\texttt{low}, \texttt{new}, \texttt{newer}, \ldots\}.$$

Después aparece en un nuevo documento:

$$\texttt{lower}.$$

Si utilizamos exclusivamente palabras completas, `lower` no pertenece al vocabulario.

$$\texttt{lower} \notin V.$$

Problema

No importa qué tan grande sea el corpus (es decir la *colección de textos*) de entrenamiento: aparecerán palabras que el modelo no observó durante el entrenamiento.

Una solución moderna consiste en trabajar también con unidades **más pequeñas que una palabra**: las **subpalabras**.

Tokenización

La **tokenización** transforma una cadena de texto en una secuencia de unidades que el modelo puede procesar:

$$\text{texto} \longrightarrow [t_1, t_2, \ldots, t_n].$$

Tokenización

La **tokenización** transforma una cadena de texto en una secuencia de unidades que el modelo puede procesar:

$$\text{texto} \longrightarrow [t_1, t_2, \ldots, t_n].$$

Los tokens pueden corresponder a:

- palabras completas,
- partes frecuentes de palabras (**subwords**),
- y, en casos extremos, caracteres individuales.

Tokenización

La **tokenización** transforma una cadena de texto en una secuencia de unidades que el modelo puede procesar:

$$\text{texto} \longrightarrow [t_1, t_2, \ldots, t_n].$$

Los tokens pueden corresponder a:

- palabras completas,
- partes frecuentes de palabras (**subwords**),
- y, en casos extremos, caracteres individuales.

Por ejemplo, una palabra no observada puede representarse mediante unidades conocidas:

$$\texttt{lower} \longrightarrow [\texttt{low}, \texttt{er}].$$

Ventaja

Un vocabulario de subpalabras permite representar palabras nuevas combinando tokens que el modelo ya conoce.

Byte-Pair Encoding (BPE)

Un algoritmo ampliamente utilizado para construir un vocabulario de subpalabras es **Byte-Pair Encoding (BPE)**.

Byte-Pair Encoding (BPE)

Un algoritmo ampliamente utilizado para construir un vocabulario de subpalabras es **Byte-Pair Encoding (BPE)**.

BPE puede entenderse en dos etapas:

1. **Entrenamiento:** a partir de un corpus, se aprende un vocabulario de tokens.
2. **Codificación:** un nuevo texto se divide utilizando el vocabulario aprendido.

Byte-Pair Encoding (BPE)

Un algoritmo ampliamente utilizado para construir un vocabulario de subpalabras es **Byte-Pair Encoding (BPE)**.

BPE puede entenderse en dos etapas:

1. **Entrenamiento:** a partir de un corpus, se aprende un vocabulario de tokens.
2. **Codificación:** un nuevo texto se divide utilizando el vocabulario aprendido.

$$\boxed{\text{corpus de entrenamiento} \xrightarrow{\text{BPE trainer}} V}$$

$$\boxed{\text{nuevo texto} + V \xrightarrow{\text{BPE encoder}} [t_1, t_2, \ldots, t_n]}$$

La idea del entrenamiento es muy simple: **combinar repetidamente pares de tokens frecuentes**.

BPE: ejemplo de entrenamiento

Supongamos que el corpus contiene inicialmente los caracteres:

A B D C A B E C A B

con vocabulario

$$V = \{A, B, C, D, E\}.$$

BPE: ejemplo de entrenamiento

Supongamos que el corpus contiene inicialmente los caracteres:

`A B D C A B E C A B`

con vocabulario

$$V = \{A, B, C, D, E\}.$$

El par adyacente más frecuente es `A B`. Lo combinamos:

`AB D C AB E C AB`

$$V = \{A, B, C, D, E, AB\}.$$

BPE: ejemplo de entrenamiento

Supongamos que el corpus contiene inicialmente los caracteres:

`A B D C A B E C A B`

con vocabulario

$$V = \{A, B, C, D, E\}.$$

El par adyacente más frecuente es `A B`. Lo combinamos:

`AB D C AB E C AB`

$$V = \{A, B, C, D, E, AB\}.$$

Ahora el par más frecuente es `C AB`:

`AB D CAB E CAB`

$$V = \{A, B, C, D, E, AB, CAB\}.$$

BPE: ejemplo de entrenamiento

Supongamos que el corpus contiene inicialmente los caracteres:

`A B D C A B E C A B`

con vocabulario

$$V = \{A, B, C, D, E\}.$$

El par adyacente más frecuente es `A B`. Lo combinamos:

`AB D C AB E C AB`

$$V = \{A, B, C, D, E, AB\}.$$

Ahora el par más frecuente es `C AB`:

`AB D CAB E CAB`

$$V = \{A, B, C, D, E, AB, CAB\}.$$

El procedimiento continúa hasta realizar k fusiones, es decir hasta generar k tokens nuevos.

BPE: algoritmo conceptual

1. Inicializar el vocabulario con unidades pequeñas, por ejemplo caracteres.

BPE: algoritmo conceptual

1. Inicializar el vocabulario con unidades pequeñas, por ejemplo caracteres.
2. Contar pares de tokens adyacentes en el corpus.

BPE: algoritmo conceptual

1. Inicializar el vocabulario con unidades pequeñas, por ejemplo caracteres.
2. Contar pares de tokens adyacentes en el corpus.
3. Encontrar el par más frecuente.

BPE: algoritmo conceptual

1. Inicializar el vocabulario con unidades pequeñas, por ejemplo caracteres.
2. Contar pares de tokens adyacentes en el corpus.
3. Encontrar el par más frecuente.
4. Fusionar ese par y agregar el nuevo token al vocabulario.

BPE: algoritmo conceptual

1. Inicializar el vocabulario con unidades pequeñas, por ejemplo caracteres.
2. Contar pares de tokens adyacentes en el corpus.
3. Encontrar el par más frecuente.
4. Fusionar ese par y agregar el nuevo token al vocabulario.
5. Repetir hasta alcanzar el número deseado de fusiones.

Así obtenemos unidades de diferentes tamaños:

caracteres $\longrightarrow$ subpalabras frecuentes $\longrightarrow$ algunas palabras completas.

Interpretación

BPE aprende el vocabulario **a partir de las frecuencias observadas en el corpus**.

BPE: los merges ocurren dentro de las palabras[3]

En la práctica, BPE no trabaja sobre todo el texto como una única secuencia de caracteres.

Primero se separa el corpus en palabras:

`set ␣new ␣new ␣renew ␣reset ␣renew`

Las frecuencias se calculan utilizando todo el corpus, pero:

los merges no pueden cruzar las fronteras entre palabras.

[3] Jurafsky y Martin, Sec. 2.4.1, ejemplo (2.11).

Ejemplo

Partimos del corpus:

`set ␣new ␣new ␣renew ␣reset ␣renew`

Agrupamos las palabras iguales y contamos sus apariciones:

Frecuencia	Palabra
2	␣n e w
2	␣r e n e w
1	s e t
1	␣r e s e t

El vocabulario inicial contiene:

$$V = \{␣, \mathtt{e}, \mathtt{n}, \mathtt{r}, \mathtt{s}, \mathtt{t}, \mathtt{w}\}.$$

¿Qué pares puede considerar BPE?

Por ejemplo, en

$$\texttt{␣n e w}$$

los pares adyacentes son

$$(\texttt{␣}, \texttt{n}), \qquad (\texttt{n}, \texttt{e}), \qquad (\texttt{e}, \texttt{w}).$$

Mientras que en

$$\texttt{␣r e n e w}$$

tenemos

$$(\texttt{␣}, \texttt{r}), \quad (\texttt{r}, \texttt{e}), \quad (\texttt{e}, \texttt{n}), \quad (\texttt{n}, \texttt{e}), \quad (\texttt{e}, \texttt{w}).$$

Pero nunca se consideran pares entre **dos palabras distintas**.

Primer merge

Contamos los pares dentro de las palabras.
El par más frecuente es

$$(\mathtt{n}, \mathtt{e}).$$

Aparece en:

$$\mathtt{␣n\ e\ w} \qquad \text{(frecuencia 2)}$$

y en

$$\mathtt{␣r\ e\ n\ e\ w} \qquad \text{(frecuencia 2)}.$$

Por tanto,

$$\mathtt{count}(\mathtt{n\ e}) = 2 + 2 = 4.$$

Creamos un nuevo token:

$$\boxed{\mathtt{n} + \mathtt{e} \longrightarrow \mathtt{ne}}$$

Corpus después del primer merge

Después de

$$\texttt{n} + \texttt{e} \longrightarrow \texttt{ne},$$

tenemos:

Frecuencia	Palabra
2	␣ne w
2	␣r e ne w
1	s e t
1	␣r e s e t

El nuevo vocabulario contiene ahora el token ne

$$V = \{\text{␣}, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}\}.$$

Segundo merge

Volvemos a contar los pares. Ahora el más frecuente es

$$(\texttt{ne}, \texttt{w}),$$

con frecuencia total

$$4.$$

Por tanto:

$$\boxed{\texttt{ne} + \texttt{w} \longrightarrow \texttt{new}}$$

y obtenemos:

Frecuencia	Palabra
2	␣new
2	␣r e new
1	s e t
1	␣r e s e t

$$V = \{␣, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}, \texttt{new}\}.$$

El espacio también puede formar parte de un token

Después de aprender `new`, el siguiente par más frecuente es

$$(\text{␣}, \texttt{r}),$$

que aparece al inicio de

$$\text{␣}\texttt{r e new} \quad \texttt{y} \quad \text{␣}\texttt{r e s e t}.$$

Se realiza entonces el merge

$$\boxed{\text{␣} + \texttt{r} \longrightarrow \text{␣}\texttt{r}}$$

El espacio puede formar parte de un token, pero sigue indicando el **inicio de una palabra**.

$$V = \{\text{␣}, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}, \texttt{new}, \text{␣}\texttt{r}\}.$$

Aprendiendo el prefijo `re-`

El siguiente merge es

$$(\text{␣r}, \text{e}) \longrightarrow \text{␣re}.$$

Entonces obtenemos:

Frecuencia	Palabra
2	␣new
2	␣re new
1	s e t
1	␣re s e t

El sistema ha aprendido esencialmente una unidad

$$\boxed{\text{␣re}}$$

que aparece al comienzo de distintas palabras.

$$V = \{\text{␣}, \text{e}, \text{n}, \text{r}, \text{s}, \text{t}, \text{w}, \text{ne}, \text{new}, \text{␣r}, \text{␣re}\}.$$

Si continuamos...

Si continuamos los merges, obendríamos

$$V = \{\text{␣}, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}, \texttt{new}, \text{␣}\texttt{r}, \text{␣}\texttt{re}, \text{␣}\texttt{new}, \text{␣}\texttt{renew}, \texttt{se}, \texttt{set}\}.$$

La idea importante

El espacio inicial

␣

sí puede participar en los merges de su palabra.
Por ejemplo:

$$␣ + \texttt{r} + \texttt{e} \longrightarrow ␣\texttt{re}.$$

Pero las palabras siguen siendo unidades separadas:

set	␣new	␣renew	␣reset

BPE no realiza merges entre dos palabras distintas.

BPE aprende subpalabras reutilizables

En el ejemplo, BPE aprende progresivamente tokens como

$$\texttt{ne}, \qquad \texttt{new}, \qquad \texttt{␣r}, \qquad \texttt{␣re}.$$

Así,

$$\texttt{␣renew}$$

puede representarse temporalmente como

$$\boxed{\texttt{␣re} + \texttt{new}.}$$

Los conteos provienen del corpus completo, pero

los merges se restringen al interior de cada palabra.

BPE encoder

Una vez aprendido el vocabulario, el **BPE encoder** se utiliza para tokenizar texto nuevo.

1. Se separa cada palabra en caracteres.
2. Se aplican los **merges aprendidos durante el entrenamiento**, en el mismo orden en que fueron aprendidos.

Por ejemplo, si durante el entrenamiento aprendimos:

$$\texttt{n} + \texttt{e} \longrightarrow \texttt{ne},$$

$$\texttt{ne} + \texttt{w} \longrightarrow \texttt{new},$$

el encoder aplica primero:

$$\texttt{n} \;\; \texttt{e} \longrightarrow \texttt{ne},$$

y después:

$$\texttt{ne} \;\; \texttt{w} \longrightarrow \texttt{new}.$$

Importante: Las frecuencias del texto nuevo **no intervienen**, se utilizan los merges y su orden aprendidos con el corpus de entrenamiento.

BPE encoder: palabras no vistas

Los merges pueden capturar unidades que aparecen después en **palabras no vistas durante el entrenamiento**.

Por ejemplo, el entrenamiento anterior aprendió:

`␣re`

como una unidad asociada al prefijo `re-`.
Puede reutilizarse posteriormente en palabras como

`␣revisit` o `␣rearrange`.

También aprendió el token `new` sin un espacio inicial, por lo que puede aparecer **dentro de otras palabras**, por ejemplo:

`␣a new`.

Así, las palabras desconocidas pueden representarse mediante **varios tokens conocidos**.

Corpus

Un **corpus** es una colección de textos (o transcripciones de habla) utilizada para estudiar o modelar el lenguaje.

Corpus

Un **corpus** es una colección de textos (o transcripciones de habla) utilizada para estudiar o modelar el lenguaje.

Pero los textos de un corpus no son una muestra neutral del "lenguaje en general". Pueden variar por:

- idioma y variedad lingüística,
- género: noticias, libros, redes sociales, artículos científicos, etc.,
- contexto: conversación, texto editado, notas médicas, documentos legales, etc.,
- características de quienes producen el texto,
- periodo histórico.

Corpus

Un **corpus** es una colección de textos (o transcripciones de habla) utilizada para estudiar o modelar el lenguaje.

Pero los textos de un corpus no son una muestra neutral del "lenguaje en general". Pueden variar por:

- idioma y variedad lingüística,
- género: noticias, libros, redes sociales, artículos científicos, etc.,
- contexto: conversación, texto editado, notas médicas, documentos legales, etc.,
- características de quienes producen el texto,
- periodo histórico.

Consecuencia para Machine Learning

El corpus de entrenamiento determina qué patrones puede aprender el modelo y qué tan bien podrá generalizar a nuevos textos.

¿De dónde vienen nuestros datos?

Antes de entrenar un modelo con texto conviene preguntar:

- ¿Por qué y para qué se recolectó el corpus?
- ¿Quién produjo los textos y en qué contexto?
- ¿Qué idioma, región o variedad lingüística contiene?
- ¿Cómo se seleccionaron los documentos?
- ¿Qué preprocesamiento se realizó?
- ¿Existen anotaciones? ¿Cómo se obtuvieron?

¿De dónde vienen nuestros datos?

Antes de entrenar un modelo con texto conviene preguntar:

- ¿Por qué y para qué se recolectó el corpus?
- ¿Quién produjo los textos y en qué contexto?
- ¿Qué idioma, región o variedad lingüística contiene?
- ¿Cómo se seleccionaron los documentos?
- ¿Qué preprocesamiento se realizó?
- ¿Existen anotaciones? ¿Cómo se obtuvieron?

Se recomienda documentar estas características mediante *datasheets* o *data statements*.

Idea importante

Un cambio entsre la distribución del corpus de entrenamiento y la de los textos reales puede producir una pérdida importante de desempeño.

Preprocesamiento con expresiones regulares

Una **expresión regular** (regex) describe un patrón de caracteres.

En procesamiento de textos puede utilizarse para:

- buscar patrones,
- identificar determinados tipos de cadenas,
- sustituir o normalizar fragmentos de texto,
- apoyar la etapa de **pre-tokenización**.

Preprocesamiento con expresiones regulares

Una **expresión regular** (regex) describe un patrón de caracteres.
En procesamiento de textos puede utilizarse para:

- buscar patrones,
- identificar determinados tipos de cadenas,
- sustituir o normalizar fragmentos de texto,
- apoyar la etapa de **pre-tokenización**.

Por ejemplo, en Python:

```
re.search(pattern, string)
```

busca la primera coincidencia del patrón dentro de una cadena.

Preprocesamiento con expresiones regulares

Una **expresión regular** (regex) describe un patrón de caracteres.
En procesamiento de textos puede utilizarse para:

- buscar patrones,
- identificar determinados tipos de cadenas,
- sustituir o normalizar fragmentos de texto,
- apoyar la etapa de **pre-tokenización**.

Por ejemplo, en Python:

```
re.search(pattern, string)
```

busca la primera coincidencia del patrón dentro de una cadena.

En este curso

No estudiaremos regex exhaustivamente. Nos interesan como una herramienta práctica para preparar texto antes de construir las representaciones numéricas.

¿Qué significa “limpiar” un texto?

Dependiendo del problema, el preprocesamiento puede incluir decisiones como:

- convertir o no a minúsculas,
- tratar signos de puntuación,
- normalizar espacios,
- identificar URLs, números u otros patrones,
- separar el texto en unidades apropiadas para la tarea.

¿Qué significa “limpiar” un texto?

Dependiendo del problema, el preprocesamiento puede incluir decisiones como:

- convertir o no a minúsculas,
- tratar signos de puntuación,
- normalizar espacios,
- identificar URLs, números u otros patrones,
- separar el texto en unidades apropiadas para la tarea.

Ejemplo:

```
¡Machine Learning es GENIAL!!!
```

podría transformarse en

```
machine learning es genial
```

¿Qué significa “limpiar” un texto?

Dependiendo del problema, el preprocesamiento puede incluir decisiones como:

- convertir o no a minúsculas,
- tratar signos de puntuación,
- normalizar espacios,
- identificar URLs, números u otros patrones,
- separar el texto en unidades apropiadas para la tarea.

Ejemplo:

```
¡Machine Learning es GENIAL!!!
```

podría transformarse en

```
machine learning es genial
```

Pero cuidado

No existe un preprocesamiento universalmente correcto.

Del texto a los datos

Al terminar esta etapa tenemos una secuencia de unidades discretas:

"El modelo aprende del texto"

$$\Downarrow$$

$$[\texttt{el}, \texttt{modelo}, \texttt{aprende}, \texttt{del}, \texttt{texto}]$$

Del texto a los datos

Al terminar esta etapa tenemos una secuencia de unidades discretas:

"El modelo aprende del texto"

$$\Downarrow$$

$$[\texttt{el}, \texttt{modelo}, \texttt{aprende}, \texttt{del}, \texttt{texto}]$$

Pero los algoritmos de Machine Learning todavía necesitan números:

$$[t_1, t_2, \ldots, t_n] \quad \longrightarrow \quad x \in \mathbb{R}^p.$$

Del texto a los datos

Al terminar esta etapa tenemos una secuencia de unidades discretas:

"El modelo aprende del texto"

$$\Downarrow$$

$$[\texttt{el}, \texttt{modelo}, \texttt{aprende}, \texttt{del}, \texttt{texto}]$$

Pero los algoritmos de Machine Learning todavía necesitan números:

$$[t_1, t_2, \ldots, t_n] \quad \longrightarrow \quad x \in \mathbb{R}^p.$$

Siguiente tema: 4.2 Representación numérica de textos

Estudiaremos cómo transformar documentos en vectores

¡Gracias!